

METAMATEMÁTICO / NLE v7.0

Reporte Técnico Exhaustivo del Sistema

Arquitectura, componentes, conexiones, flujos y funcionamiento detallado

Generado a partir de análisis directo del código fuente
Repositorio: E:\Metamatematico\

18 de agosto de 2026

Abstract

Este documento describe, con precisión técnica y a nivel de archivo/línea, la totalidad del sistema **NLE v7.0** (Núcleo Lógico Evolutivo): una IA matemática que combina un grafo categórico de habilidades, un sistema de memoria evolutiva (MES, Ehresmann), aprendizaje por refuerzo (GNN+PPO), verificación formal en Lean 4/Mathlib y modelos de lenguaje (Claude y otros proveedores), expuesta mediante una aplicación Streamlit. El reporte cubre cada subsistema de `nucleo/`, la capa de formalización Lean (`MetamathProver/`), la interfaz de usuario (`app.py`, `pages/`), la infraestructura de despliegue y los scripts de entrenamiento/datos, documentando explícitamente qué partes están **activas en producción**, cuáles son **parciales/heurísticas** y cuáles están **latentes o desconectadas** del camino de ejecución real.

Contents

1	Visión general de la arquitectura	4
1.1	Qué es el sistema	4
1.2	Los tres consumidores de <code>Nucleo</code>	4
1.3	Mapa de subsistemas de <code>nucleo/</code>	4
1.4	Diagrama textual del flujo principal	5
2	Núcleo orquestador: <code>nucleo/core.py</code>, <code>cli.py</code>, <code>config.py</code>, <code>types.py</code>	7
2.1	<code>nucleo/types.py</code> — capa de datos pura	7
2.1.1	Enumeraciones	7
2.1.2	Dataclasses principales	7
2.2	<code>nucleo/config.py</code> — esquema de configuración	8
2.3	<code>nucleo/core.py</code> — la clase <code>Nucleo</code> (2623 líneas)	8
2.3.1	Clases auxiliares	8
2.3.2	Atributos clave de <code>Nucleo</code>	8
2.3.3	<code>initialize()</code> — construcción del grafo de dependencias	9
2.3.4	<code>process(input_text) -> NucleoResponse</code> — el punto de entrada único	9
2.3.5	<code>_math_via_lean(input_text)</code> — el pipeline “Lean-first”	10
2.3.6	Otros métodos públicos relevantes	10
2.3.7	<code>_is_mathematical(text)</code> — el clasificador de enrutamiento	11
2.4	<code>nucleo/cli.py</code> — interfaz de terminal	11
3	Grafo categórico de habilidades: <code>nucleo/graph/</code>	12
3.1	<code>category.py</code> — la categoría <code>SkillCategory</code> (1153 líneas)	12
3.1.1	Estructura de almacenamiento	12
3.1.2	Operaciones categóricas	12
3.1.3	Consultas jerárquicas (MES v7.0)	12
3.1.4	Clasificación de enlaces y axiomas formales	12
3.1.5	Puntuaciones de recompensa	12
3.2	<code>complexity.py</code> — jerarquía emergente $c_n(X)$	13
3.3	<code>embeddings.py</code> — representaciones vectoriales	13
3.4	<code>evolution.py</code> — <code>EvolutionarySystem</code> (663 líneas)	13
3.5	<code>lean_proof_generator.py</code> — puente Python→Lean para el propio grafo	14
3.6	<code>operations.py</code> — operaciones atómicas (Def. 4.3)	14
4	Sistema de Memoria Evolutiva (MES): <code>nucleo/mes/</code>	15
4.1	Visión general	15
4.2	<code>co_regulators.py</code> — la red de co-reguladores (1209 líneas)	15
4.2.1	Ciclo de 4 fases de todo <code>CoRegulator</code>	15
4.2.2	<code>TacticalCoRegulator</code>	15
4.2.3	<code>OrganizationalCoRegulator</code>	16
4.2.4	<code>StrategicCoRegulator</code>	16

4.2.5	IntegrityCoRegulator	16
4.2.6	CoRegulatorNetwork.decide(query, grafo) — el protocolo de transición global	16
4.3	memory.py — arquitectura de memoria de 4 capas (707 líneas)	16
4.4	patterns.py — patrones y colimites (1197 líneas)	17
4.4.1	PatternManager	17
4.4.2	ColimitBuilder	17
5	Aprendizaje por refuerzo: nucleo/rl/	18
5.1	Formulación del MDP (mdp.py)	18
5.2	gnn.py — codificador GNN (SkillGNN)	18
5.3	networks.py — ActorCriticNetwork	18
5.4	agent.py — NucleoAgent y PPO (565 líneas)	19
5.4.1	Selección de acción: tres niveles	19
5.4.2	Actualización PPO con GAE	19
5.4.3	Persistencia	19
5.5	rewards.py — función de recompensa estructural (334 líneas)	19
5.6	Consumidores de nucleo/rl/	19
6	Integración con Lean 4: nucleo/lean/ y MetamathProver/	20
6.1	client.py — invocación de Lean y LeanResult	20
6.1.1	Mecanismo de invocación	20
6.1.2	Fallback HTTP	20
6.1.3	LeanResult	20
6.1.4	Manejo de timeouts y correcciones específicas de Windows	20
6.1.5	Normalización de código antes de cada invocación (_normalize_code)	20
6.2	solver_cascade.py — la cascada de solucionadores	21
6.3	sorry_analyzer.py y sorry_filler.py	21
6.4	tactics.py / tactics_db.py — dos catálogos de tácticas paralelos	21
6.5	parser.py — interpretación de salida/errores de Lean	22
6.6	Formalizaciones Lean: MetamathProver/	22
6.6.1	Toolchain	22
6.6.2	CategoryFoundations/ — fundamentos formales de la propia arquitectura del NLE	22
6.6.3	Group/ y Ring/ — álgebra clásica (0 sorry confirmado en todo el árbol)	22
6.7	Servicio remoto: lean_api/main.py	23
7	Pilares fundacionales: nucleo/pillars/	24
7.1	base.py — la interfaz Pillar	24
7.2	Los cuatro pilares concretos	24
7.3	math_domains.py — el catálogo real (162 habilidades)	24
8	Integración con modelos de lenguaje: nucleo/llm/	26
8.1	client.py — cliente multi-proveedor (529 líneas)	26
8.2	prompts.py — catálogo de plantillas (314 líneas)	26
9	Evaluación de respuestas: nucleo/eval/	27
9.1	math_evaluator.py (492 líneas)	27
9.1.1	Extracción (extract_answer)	27
9.1.2	Normalización (_normalize_string)	27
9.1.3	Comparación (_compare_answers) — primera coincidencia gana	27
10	Consultores Avanzados: nucleo/consultores/	28

10.1	Propósito	28
10.2	Protocolo de marcadores (<code>master_prompt.py</code>)	28
10.3	<code>orchestrator.py</code> — <code>ConsultoresModule.process(query)</code>	28
10.4	Fórmula de puntuación (<code>artifacts.py::CandidateMetrics.compute</code>)	29
10.5	<code>solver_runner.py</code> — ejecución aislada	29
10.6	Activación	29
11	Sistema multi-agente: <code>nucleo/multi_agent/</code>	30
11.1	Propósito y estado real: LATENTE	30
11.2	<code>specialized_agent.py</code> — <code>SpecializedAgent</code>	30
11.3	<code>orchestrator.py</code> — <code>MultiAgentOrchestrator</code>	30
11.4	<code>mes_bridge.py</code> — <code>MESBridge</code> , el puente hacia <code>nucleo/mes/</code>	30
11.5	<code>pillar_agents.py</code> y <code>colimit_agents.py</code> — jerarquía categórica $L0 \rightarrow L1 \rightarrow L2 \rightarrow L3$	31
11.6	<code>scripts/train_multiagent.py</code> — productor de los pesos	31
12	Interfaz de usuario: <code>app.py</code> y <code>pages/</code>	32
12.1	<code>app.py</code> — estructura general (1664 líneas)	32
12.1.1	Arranque	32
12.1.2	Instanciación del Nucleo (punto clave)	32
12.1.3	Flujo de chat: entrada del usuario $\rightarrow$ <code>Nucleo</code> $\rightarrow$ <code>render</code>	32
12.1.4	Métodos de <code>Nucleo</code> usados por la UI (superficie pública real)	33
12.1.5	Navegación multipágina	33
12.2	Páginas	33
13	Infraestructura y despliegue	34
13.1	<code>Dockerfile</code>	34
13.2	<code>deploy/</code>	34
13.3	Arranque local en Windows (autoarranque)	34
13.4	<code>nucleo_config.yaml</code>	34
14	Scripts de entrenamiento y pipeline de datos	35
14.1	Pipeline de datos	35
14.2	Entrenamiento	35
14.3	Evaluación y herramientas	35
15	Flujo end-to-end de una consulta matemática	36
16	Estado real del sistema: honestidad y limitaciones conocidas	37
17	Conclusión	39

Chapter 1

Visión general de la arquitectura

1.1 Qué es el sistema

NLE v7.0 es una aplicación Python (~12 800 LOC en `nucleo/`) más un pequeño núcleo de formalización en Lean 4 (14 archivos `.lean`), organizada en torno a una clase orquestadora única, `Nucleo` (`nucleo/core.py`), que implementa el sistema dinámico

$$\Sigma_t = (L, CR_t, G_t, F)$$

donde L es el cliente LLM, CR_t es una red de cuatro *co-reguladores* (táctico, organizacional, estratégico, de integridad) que sustituye a un agente único de RL, G_t es el grafo categórico de habilidades en el instante t , y $F = \{F_{Set}, F_{Cat}, F_{Log}, F_{Type}\}$ son los cuatro “pilares” fundacionales (teoría de conjuntos, categorías, lógica, tipos). El diseño se basa explícitamente en los *Memory Evolutive Systems* de Ehresmann y Vanbremeersch.

1.2 Los tres consumidores de Nucleo

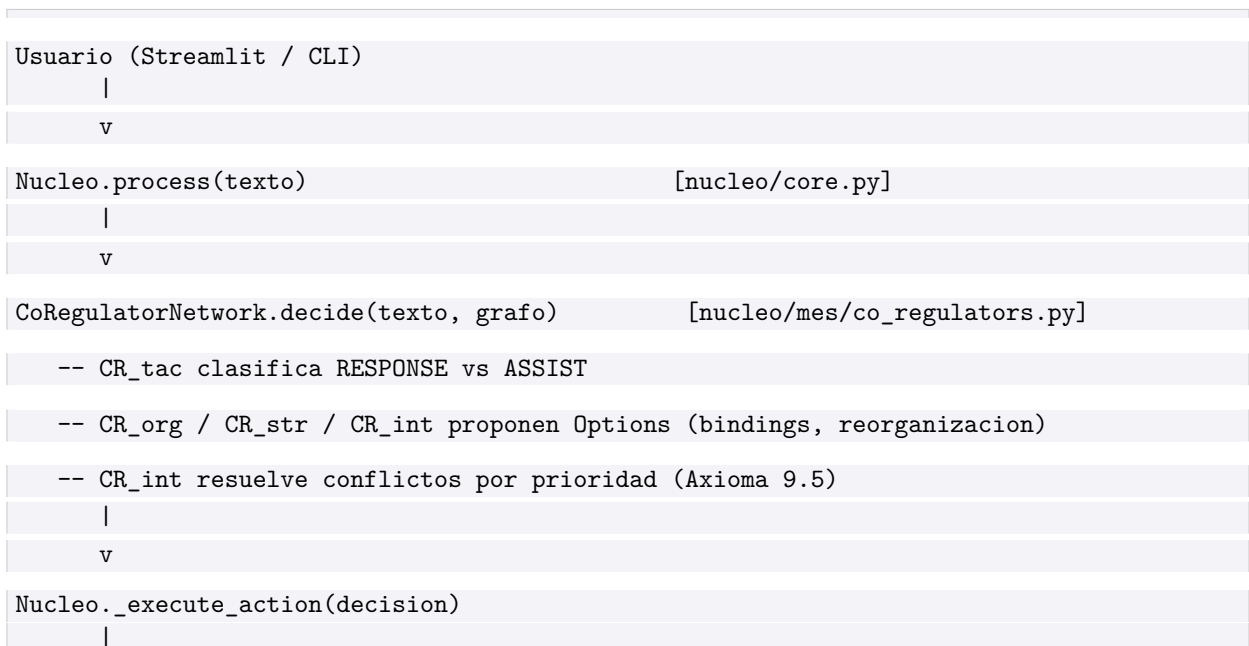
- **Interfaz web** (`app.py` + `pages/*.py`, Streamlit): el punto de entrada de producción, para usuarios finales.
- **CLI** (`nucleo/cli.py`, comando `python -m nucleo chat`): REPL de terminal con Rich, para desarrollo/depuración.
- **Scripts de entrenamiento/evaluación** (`scripts/*.py`): instancian `Nucleo` programáticamente para benchmarking (MATH/GSM8K) o entrenan las redes GNN+PPO fuera de línea.

1.3 Mapa de subsistemas de `nucleo/`

Subsistema	Estado	Rol
<code>core.py</code>	ACTIVO	Orquestador central; punto de entrada único (<code>process()</code>).
<code>graph/</code>	ACTIVO	Grafo categórico de habilidades (objetos= <code>Skill</code> , morfismos= <code>Morphism</code>); evolución temporal, jerarquía emergente, embeddings.

Subsistema	Estado	Rol
mes/	ACTIVO	Memoria Evolutiva: 4 co-reguladores, memoria procedimental/semántica, patrones/colimites. Sustituye al agente único de RL como tomador de decisiones.
rl/	PARCIAL	GNN (GATConv) + PPO Actor-Crítico; alimentado con aprendizaje en vivo, pero el enrutamiento por defecto usa heurísticas porque el agente entrenado colapsó a una única acción (ver §16).
lean/	ACTIVO	Cliente Lean 4 (subprocess/HTTP), cascada de solucionadores, análisis/relleno de <code>sorry</code> , catálogo de tácticas, parser de errores. Lean es la única fuente de verdad matemática.
pillars/	MIXTO	4 clases <code>Pillar</code> formales (ZFC/Cat/Lógica/Tipos) usadas solo por <code>cli.py init</code> y tests; el catálogo real que alimenta el grafo en producción es <code>math_domains.py</code> (162 habilidades).
llm/	ACTIVO	Cliente multi-proveedor (Anthropic Claude por defecto; también Google, Groq, DeepSeek); prompts de formalización/traducción/tutor.
eval/	ACTIVO	Evaluador de respuestas estilo InternLM-Math (extracción <code>\boxed{}</code> , comparación numérica/simbólica).
consultores/	ACTIVO (opt-in)	Módulo “Consultores Avanzados”: genera N candidatos verificables en una sola llamada LLM, cada uno verificado en Lean y rankeado.
multi_agent/	LATENTE	14 agentes especializados por categoría + jerarquía de “agentes colimit”; código completo y probado pero <code>set_multi_agent_orchestrator()</code> nunca se invoca en la app real.

1.4 Diagrama textual del flujo principal



```

    +-- RESPONSE, no matematico -> LLMClient.generate() (chat directo)
    |
    +-- RESPONSE / ASSIST, matematico -> Nucleo._math_via_lean()
    |
    |      +-- classify_query() + domain_default_tactic() [multi_agent/
specialized_agent.py]
    |
    |      +-- LLMClient.generate() (formaliza en Lean 4, con few-shot miniF2F)
    |
    |      +-- LeanClient.check_code() [nucleo/lean/client.
py]
    |
    |      -- local: subprocess "lake env lean --json"
    |
    |      -- fallback remoto: HTTP a lean_api (Railway)
    |
    |      +-- si SORRY -> SolverCascade / SorryFiller [nucleo/lean/*]
    |
    |      +-- LLMClient.generate() (traduce Lean verificado a lenguaje natural)
    |
    +-- REORGANIZE -> EvolutionarySystem.apply_option() [nucleo/graph/
evolution.py]
    |
    v
Nucleo._record_experience() -> MESMemory + Transition PPO en vivo
Nucleo._apply_structural_evolution() -> aplica Options pendientes al grafo
    |
    v
NucleoResponse(content, lean_result, confidence, metadata)
    |
    v
app.py: render en chat, boton feedback (+1.0/-0.5) -> nucleo.apply_feedback()

```


Chapter 2

Núcleo orquestador: `nucleo/core.py`, `cli.py`, `config.py`, `types.py`

2.1 `nucleo/types.py` — capa de datos pura

Sin dependencias de otros submódulos de `nucleo`. Define el vocabulario compartido:

2.1.1 Enumeraciones

- `MorphismType`: `IDENTITY`, `DEPENDENCY`, `SPECIALIZATION`, `ANALOGY`, `TRANSLATION`.
- `ActionType`: `RESPONSE`, `REORGANIZE`, `ASSIST` — los tres tipos base de acción, $A = A_1 \sqcup A_2 \sqcup A_3$.
- `PillarType`: `SET`, `CAT`, `LOG`, `TYPE`.
- `CoRegulatorType`: `TACTICAL`, `ORGANIZATIONAL`, `STRATEGIC`, `INTEGRITY`.
- `MESActionType`: acción extendida $A' = A_1|A_2|A_3|A_4$, añade `COMPLEXIFY`, `FORM_CONCEPT`, `CREATE_LEVEL`, `REPAIR_FRACTURE`.
- `MemoryType`: `PROCEDURAL`, `SEMANTIC`, `CONSOLIDATED`, `EMPIRICAL`.
- `FractureType`: `TEMPORAL`, `STRUCTURAL`, `LOGICAL`.
- `LinkComplexity`: `IDENTITY`, `SIMPLE`, `COMPLEX` — `COMPLEX` define emergencia (Teorema 8.6).

2.1.2 Dataclasses principales

- `Skill`: unidad de conocimiento / objeto categórico. Campos: `id`, `name`, `description`, `pillar`, `level`, `content`, `metadata`, `status`, `pattern_ids`.
- `Morphism`: arista tipada entre `Skills`, con `weight`.
- `State`: estado del MDP $x = (c_L, g_{Lean}, G, h, m)$ — contexto LLM, meta Lean activa, snapshot del grafo, historial, métricas.
- `Action`: `action_type` + `params`; constructores de conveniencia `Action.response/reorganize/assist`.
- `RewardComponents`: descomposición de recompensa (Ec. 5) con método `total( $\lambda_1.. \lambda_5$ )`.
- `Pattern`: funtor $P : I \rightarrow K$ (Def. 2.1) — `component_ids`, `distinguished_links`, datos opcionales de funtor completo.
- `Colimit`: objeto colímite de un patrón (Def. 2.2) — morfismos de co-cono, verificación de

propiedad universal.

- **ExperienceRecord**: registro de memoria (Def. 5.2), con `consolidate()` que promueve a CONSOLIDATED tras 3 consolidaciones.
- **EConcept**: concepto semántico E (Def. 5.3), clase de equivalencia de registros.
- **Fracture**: fractura del sistema (Def. 7.4).
- **Option**: opción de complejización (Def. 2.9) — `absorptions`, `eliminations`, `bindings`, `classifications`; es el tipo que `Nucleo._apply_structural_evolution` consume.

2.2 nucleo/config.py — esquema de configuración

Jerarquía de `dataclasses` sin dependencias de otros submódulos:

- **RLConfig**: $\gamma = 0.99$, $\lambda_1 = 0.1$ (eficiencia), $\lambda_2 = 0.05$ (organización), $\lambda_3 = 0.2$ (emergencia), $\lambda_4 = 0.1$ (jerarquía, v7.0), $\lambda_5 = 0.08$ (memoria, v7.0), `learning_rate=3e-4`, `batch_size=64`, `clip_range=0.2`.
- **LeanConfig**: `lean_path="lake"`, `timeout_ms=360000` (6 min — comentario explícito: Mathlib en disco USB puede tardar 167–282s), `use_mathlib=True`.
- **GraphConfig**: `max_skills=100000`, `gnn_hidden_dim=256`, `gnn_num_layers=3`.
- **MESConfig**: `cr_org_frequency=10` (k), `cr_str_frequency=100` (K), `cr_int_period=50`, `max_records=10000`, `consolidation_threshold=3`, `colimit_threshold=0.8`.
- **LLMConfig**: `model="claude-sonnet-4-20250514"` por defecto (el YAML de despliegue usa "claude-sonnet-5"), `max_tokens=4096`, `api_key` con fallback a `ANTHROPIC_API_KEY`.
- **NucleoConfig**: agregador; `from_yaml/to_dict/save_yaml`. **Nota de precisión**: existe una asimetría menor de ida-y-vuelta — `to_dict()` omite `fracture_check_interval`, `max_repair_attempts` y `live_learning_autosave` del diccionario serializado.

2.3 nucleo/core.py — la clase Nucleo (2623 líneas)

Éste es el módulo más grande e importante del sistema. Contiene:

2.3.1 Clases auxiliares

- **NucleoMode(Enum)**: `INTERACTIVE`, `AUTONOMOUS`, `VERIFICATION`.
- **NucleoState**: estado interno $x = (c_L, g_{Lean}, G, h, m)$; método `to_state(graph)` lo convierte al `State` del MDP.
- **NucleoResponse**: objeto de respuesta — `content`, `action_type`, `lean_result`, `suggested_skills`, `confidence`, `metadata`.

2.3.2 Atributos clave de Nucleo

Instancia y comparte entre sí: `_graph` (`SkillCategory`), `_llm` (`LLMClient`), `_lean` (`LeanClient`), `_evolution` (`EvolutionarySystem`), `_pattern_manager/_colimit_builder` (`PatternManager/ColimitBuild`), `MES`), `_memory` (`MESMemory`), `_cr_network` (`CoRegulatorNetwork`), `_solver_cascade/_sorry_filler`, `_neural_agent` (`NucleoAgent PPO`), `_multi_agent_orchestrator` (opcional, `None` por defecto), `_consultores` (opcional, `None` por defecto), `_evaluator` (`MathEvaluator`).

2.3.3 initialize() — construcción del grafo de dependencias

Orden de instanciación (crítico para que todos los componentes MES compartan las mismas instancias de PatternManager/ColimitBuilder):

1. Construye SkillCategory y carga habilidades fundacionales (§2.3.3).
2. LLMClient (salvo que ya se haya inyectado vía reconfigure_llm).
3. LeanClient apuntando a la raíz del repo (donde vive lakefile.toml).
4. SolverCascade + SorryFiller, banco few-shot data/lean_examples.json.
5. PatternManager() → ColimitBuilder(pattern_manager) → MESMemory(...) → EvolutionarySystem → CoRegulatorNetwork(...) — las cuatro últimas comparten *las mismas* instancias de pattern_manager/colimit_builder.
6. Carga memory.json persistida (merge monótono, nunca sobrescribe).
7. Crea NucleoAgent (PPO), carga pesos entrenados (neural_agent.json/.pt) y buffer de experiencia (experience_buffer.pkl) si existen; los conecta a self y a self._cr_network.

_load_foundational_skills()

Añade **10 habilidades de nivel 0** directamente en código (no desde math_domains.py): zfc-axioms, ordinals, cat-basics, functors, nat-trans, limits, fol-deduction, fol-metatheory, cic, lean-kernel, más morfismos de dependencia/traducción/analogía entre pilares. Luego llama load_math_domains(self._graph) que añade las 162 habilidades de dominio (niveles 1–3).

2.3.4 process(input_text) -> NucleoResponse — el punto de entrada único

Flujo paso a paso:

1. Inicialización perezosa si hace falta.
2. Actualiza contexto/historial en self._state.
3. **Dinámica global:** decision = self._cr_network.decide(input_text, self._graph) — la red de 4 co-reguladores decide ActionType y source_cr (véase Capítulo 4).
4. Callback opcional on_action.
5. _execute_action(decision, input_text):
 - **RESPONSE:** _is_mathematical(texto) clasifica; si es matemático → _math_via_lean(); si no, respuesta LLM directa (o demo si no hay API key).
 - **ASSIST:** _assist_lean() — maneja código Lean pegado por el usuario, o delega a _math_via_lean().
 - **REORGANIZE:** _reorganize_graph() — acción interna, no genera contenido visible.
6. Fusiona confianza/metadata de la decisión CR.
7. success = _evaluate_result(decision, response) — heurística escalar.
8. self._cr_network.record_result(decision, success, grafo).
9. _record_experience(): registro en MESMemory, formación de E-conceptos, aprendizaje de procedimiento si éxito > 0.3, Transition PPO en vivo al agente neuronal, autoguardado

opcional, persistencia cada 20 interacciones.

10. `_apply_structural_evolution()`: aplica `Options` pendientes (bindings/eliminaciones) propuestas por `CR_org/CR_str/CR_int` después de resolver la consulta actual, para no perturbar el grafo a mitad de una respuesta.
11. Devuelve `NucleoResponse`.

2.3.5 `_math_via_lean(input_text)` — el pipeline “Lean-first”

Principio arquitectónico explícito: **Lean es la única fuente de verdad matemática; el LLM solo formaliza (escribe Lean candidato) y traduce (explica Lean verificado en lenguaje natural) — nunca “razona” libremente sobre corrección.**

1. `classify_query()` (`multi_agent/specialized_agent.py`) → área matemática; `domain_default_tactic(area)` → táctica Lean por defecto para la cascada.
2. Bifurca por intención: petición visual/geométrica pura → `_math_educational_explanation` (bypass total de Lean); revisión de documento/paper → `_review_document_for_errors`; consulta definicional (“qué es X”) → ajusta el prompt para pedir `#check/structure` en vez de una prueba.
3. **Paso 1 — el LLM formaliza**: prompt con few-shot (`_build_few_shot_context`, ejemplos de miniF2F por categoría), referencias Mathlib ancladas (`_mathlib_ref_for`, diccionario `_MATHLIB_REFS` verificado a mano), detección/reintento de pruebas triviales (`_is_trivial_lean`).
4. **Paso 2 — Lean verifica**: `await self._lean.check_code(codigo)`. Si `ERROR`, intenta `repair_imports()` y reverifica una vez.
5. Bifurca por `result.status`: `SUCCESS` → verificado; `SORRY` → **Paso 3, _try_solve_sorries** (cascada de solucionadores con táctica de dominio primero, luego `SorryFiller`); `TIMEOUT/NOT_AVAILABLE/error` → notas correspondientes.
6. **Paso 4 — el LLM traduce**: prompt distinto para consultas definicionales vs. de prueba, exige fidelidad al código Lean real.
7. Ensambla `content` con insignia de estado (verificado/parcial/pendiente/timeout) + explicación + bloque de código Lean.
8. Registra el resultado (`_record_lean_experience`) y devuelve `NucleoResponse(action_type=ASSIST, lean_result=..., metadata={verification_status, area})`.

2.3.6 Otros métodos públicos relevantes

- `reconfigure_llm(provider, model, api_key, max_tokens)`: hot-swap del proveedor/-modelo (usado por la barra lateral de Streamlit).
- `sync_conversation(history)`: sincroniza el buffer conversacional interno del LLM con el historial de la UI.
- `apply_feedback(score)`: aplica retroalimentación explícita del usuario (! / !) — actualiza el último `ExperienceRecord` y dispara una actualización PPO extra con la recompensa real.
- `evaluate_answer(prediction, reference)`: delega a `MathEvaluator`.
- `process_sync(input_text)`: wrapper síncrono de `process()` — ejecuta en un hilo dedicado con su propio event loop (compatibilidad con Streamlit), timeout 420 s.
- `get_viz_data(query)`: construye el payload para la página de Visualizaciones — no

dos/aristas del grafo, habilidades emparejadas/dependencias/tácticas, embeddings, info de colimites, última decisión CR.

- `set_multi_agent_orchestrator()` / `set_consultores_mode()` / `disable_consultores_mode()`: activadores de los subsistemas opcionales (Capítulos 10–11).

2.3.7 `_is_mathematical(text)` — el clasificador de enrutamiento

Cascada de reglas: excluye aperturas conversacionales (“hola”, “gracias”) → devuelve `True` si hay símbolos matemáticos Unicode o comandos LaTeX → devuelve `True` si hay coincidencia de tokens/subcadenas contra un gran conjunto bilingüe `_MATH_KEYWORDS`.

2.4 `nucleo/cli.py` — interfaz de terminal

Basada en `argparse` + Rich. Subcomandos: `chat` (REPL interactivo que instancia Nucleo y llama `process()` en bucle), `init` (crea `nucleo_config.yaml` y registra las 4 clases `Pillar` formales vía `PillarRegistry`), `check` (verifica la conexión con Lean con un teorema trivial), `graph/validate` (grafos de demostración con 3–5 habilidades, verifican axiomas categóricos). El REPL soporta comandos `/help`, `/stats`, `/skills`, `/axioms`, `/clear`, `/quit`. Resuelve `ANTHROPIC_API_KEY` desde variable de entorno o config, cayendo a modo *mock/demo* si no hay clave.

Chapter 3

Grafo categórico de habilidades: nucleo/graph

3.1 category.py — la categoría SkillCategory (1153 líneas)

Implementa $G_t = (S_t, Mor_t, w_t)$: una pequeña categoría jerárquica enriquecida sobre $\mathbb{R}^+$.

3.1.1 Estructura de almacenamiento

`_skills` (dict[id, SkillNode]), `_morphisms`, índices de adyacencia `_outgoing/_incoming`, `_identities`, `_by_level`, `_by_pillar`, y **críticamente** `_morphism_pairs`: dict[(src,tgt), morphism_id] — solo almacena *un* morfismo por par (origen, destino), lo que hace que la categoría sea, en la práctica, un **preorden / categoría delgada (thin category)**: distintos MorphismType entre el mismo par son etiquetas del mismo morfismo, no morfismos distintos. Esta propiedad es la base de toda la maquinaria de colímites/joins.

3.1.2 Operaciones categóricas

`hom(src,tgt)`, `identity(skill_id)`, `compose(g,f)` (compone respetando leyes de identidad; regla de tipos: mismo tipo ◦ mismo tipo = mismo tipo, mixto = DEPENDENCY). `reachable_from(source_id)`: cierre transitivo BFS sobre morfismos no-identidad. `is_preorder_leq(a,b)`: $a \leq b$ si y sólo si b es alcanzable desde a — relación fundamental usada por toda la verificación de joins/colímites en `mes/patterns.py`.

3.1.3 Consultas jerárquicas (MES v7.0)

`get_skills_at_level`, `get_skills_by_pillar`, `apply_complexity_order(cn_dict)` (sobrescribe `skill.level` desde el orden de complejidad computado), `get_atoms` (nivel 0), `get_clusters` (nivel 1), `get_colimits()` (habilidades con `pattern_ids` no vacío).

3.1.4 Clasificación de enlaces y axiomas formales

`classify_link(morphism_id)` (Def. 6.3): IDENTITY/SIMPLE (factoriza a través de un colímite/clúster)/COMPLEX (no factoriza — representa emergencia, Teorema 8.6). `verify_pillar_multiplicity()` (Prop. 3.4): ≥ 2 pilares + morfismos TRANSLATION/ANALOGY entre pilares. `verify_all_axioms()` agrega los axiomas formales 8.1–8.4 (jerarquía, multiplicidad, conectividad, cobertura).

3.1.5 Puntuaciones de recompensa

`get_hierarchy_score()`: $0.4 \cdot \text{cobertura de niveles} + 0.4 \cdot \text{forma piramidal} + 0.2 \cdot \text{ratio de colímites}$.
`get_memory_score(pattern_coverage)`: $0.4 \cdot \text{conectividad} + 0.3 \cdot \text{cobertura de patrones} + 0.3 \cdot$

diversidad de pilares. Ambos alimentan directamente los términos λ_4, λ_5 de la recompensa en `rl/rewards.py`.

3.2 `complexity.py` — jerarquía emergente $c_n(X)$

$c_n(X) = 0$ si X no es join de ningún patrón; $c_n(X) = 1 + \max(c_n(\text{componente}))$ en otro caso. **No es un nivel asignado a mano**, sino el resultado de un cómputo de punto fijo:

- `compute_complexity_order(grafo, colimit_builder)`: iteración estilo Bellman-Ford sobre todos los colimites conocidos, acotada por `len(skills)+2` iteraciones.
- `_detect_convergence_patterns`: para cada habilidad X con ≥ 2 predecesores directos distintos, crea un `Pattern` — heurística canónica de detección de puntos de join.
- `build_join_for_pattern`: reutiliza join existente, o crea uno nuevo ($\text{nivel} = \max(\text{niveles de componentes})+1$), añade morfismos de co-cono y morfismos mediadores hacia todas las cotas superiores comunes (garantizando minimalidad).
- `build_hierarchy_to_fixpoint(max_iterations=20)`: itera {detectar patrones de convergencia $\rightarrow$ construir joins} hasta punto fijo, luego llama `grafo.apply_complexity_order(cn)`.

Este módulo tiene una contraparte formalmente verificada en Lean (`ComplexityOrder.lean`, con 2 `sorry` pendientes — §6.6).

3.3 `embeddings.py` — representaciones vectoriales

`semantic_embed(text, category, level, dim=256)`: bag-of-words sobre un vocabulario matemático fijo de 64 términos (dims 0–63, normalizado L2) + one-hot de categoría $\times 5.0$ (dims 64–77) + one-hot de nivel $\times 3.0$ (dims 78–80). `SkillEmbeddingModel.embed_skill`: combina embedding textual (256d) + embedding estructural (64d, grados normalizados) + rasgos discretos (pilar, contadores de dependencias). `GraphEmbedding.embed(grafo)`: embedding de 128d de todo el grafo, usado como parte del estado del MDP.

3.4 `evolution.py` — `EvolutionarySystem` (663 líneas)

Implementa el sistema evolutivo S sobre $T = \mathbb{N}$ (Def. 3.2): una familia $\{Skill_t\}$ con funtores de transición $k_{t,t'}$.

- `CategorySnapshot`: instantánea inmutable en el tiempo t (habilidades, morfismos, estadísticas).
- `TransitionFunctor`: $k_{t,t'} : Skill_t \rightarrow Skill_{t'}$; `compose(other)` implementa composición funtorial $k_{t_1,t_3} = k_{t_2,t_3} \circ k_{t_1,t_2}$.
- `apply_option(option)`: el paso evolutivo $Skill_{t+1} = Compl(Skill_t, O_t)$ — (1) absorciones, (2) eliminaciones (`grafo.remove_skill`), (3) uniones (`colimit_builder.build_colimit` por cada patrón de `option.bindings`); detecta morfismos mediadores nuevos como efecto colateral y los registra como absorbidos; construye y guarda el `TransitionFunctor`, avanza `_current_time`.
- **Advertencia de diseño explícita en el código**: si `EvolutionarySystem` y los co-reguladores de `mes/` no comparten las mismas instancias de `pattern_manager/colimit_builder`, las uniones propuestas por un CR apuntarían a patrones invisibles para el otro

y `apply_option` las descartaría silenciosamente. `core.py` evita esto explícitamente inyectando las mismas instancias en todos los componentes.

- Emergencia (Def. 6.3, Teo. 8.6): `detect_complex_links`, `measure_emergence` (ratio de emergencia, crecimiento de complejidad).
- Teoremas formales 8.5–8.7: `verify_consistency`, `verify_emergence_growth`, `verify_coverage_preservation`.

3.5 `lean_proof_generator.py` — puente Python→Lean para el propio grafo

Genera código Lean que codifica el grafo de habilidades como `Fin n` y ejecuta Lean por subproceso para verificar, en el subgrafo finito conocido: (a) los axiomas categóricos sobre el carcaj (quiver) de habilidades, y (b) que un objeto candidato es efectivamente colímite de un diagrama finito (propiedad universal via `native_decide`). El docstring es explícito sobre lo que **no** se puede verificar así: propiedades en la categoría libre infinita, colímites en sentido universal absoluto, ni equivalencia formal con el MES de Ehresmann (es una analogía, no un isomorfismo). `verify_colimit_in_lean(...)` devuelve `verified=None` (no `False`) si Lean no está instalado localmente — honestidad explícita sobre ausencia de evidencia.

3.6 `operations.py` — operaciones atómicas (Def. 4.3)

`GraphOperations`: `add_node` $O(1)$, `add_edge` $O(1)$, `merge` $O(d_{max})$ (funde dos habilidades, hereda todos los morfismos incidentes, excluye auto-bucles), `split` $O(d(s))$ (divide una habilidad en N , duplica morfismos incidentes escalados por `distribution`, añade `ANALOGY` entre las nuevas), `reweight` $O(1)$, `connect_to_pillar` (impone el Axioma 7.2: toda habilidad debe alcanzar un pilar). El Teorema 7.4 afirma que estas operaciones preservan la validez categórica.

Chapter 4

Sistema de Memoria Evolutiva (MES): nucleo/mes/

4.1 Visión general

El subsistema MES implementa la **Dinámica Global** (Def. 4.1): cuatro co-reguladores que operan a distintos niveles/escalas temporales, con prioridad estricta $CR_{int} > CR_{str} > CR_{org} > CR_{tac}$ (Axioma 9.5). Desde v7.0, **esta red reemplazó a un agente único de RL como tomador de decisiones principal** — el agente PPO (Capítulo 5) sigue existiendo y aprendiendo en vivo, pero actúa como una señal más dentro de **TacticalCoRegulator**, no como el decisor final.

4.2 co_regulators.py — la red de co-reguladores (1209 líneas)

4.2.1 Ciclo de 4 fases de todo CoRegulator

1. `build_landscape(grafo)` -> `Landscape`: decodifica una vista parcial del sistema (Def. 2.13).
2. `select_objectives(landscape)` -> `Option`: selecciona metas.
3. `encode_procedure(option)` -> `MESActionType`: codifica.
4. `evaluate(anticipado, real)` -> `float`: evalúa tras la ejecución.

4.2.2 TacticalCoRegulator

(nivel 0–1, se activa en *cada* interacción) Decide RESPONSE vs. ASSIST. `classify_query` usa una cascada de tres niveles:

1. **Agente neuronal** (si hay ≥ 50 transiciones en buffer o pesos pre-entrenados, y no está “degenerado”).
2. **Heurística de palabras/frases clave** (ASSIST_KEYWORDS, ASSIST_PHRASES, con RESPONSE_OVERRIDE para preguntas definicionales).
3. **Coincidencia de habilidades del grafo** (solo como contexto, no redirige por sí sola).

`_neural_agent_is_degenerate()` sondea el agente con 4 consultas fijas heterogéneas (`_DEGENERACY_PROBE`); si devuelve la misma acción para las 4, se marca degenerado y se ignora. **Hallazgo documentado en el propio código (2026-08-05): los pesos entrenados colapsaron a devolver siempre ASSIST**, por lo que en la práctica el enrutamiento cae casi siempre a la heurística de palabras clave (§16).

4.2.3 OrganizationalCoRegulator

(nivel 1–2, cada 10 pasos) `record_activation(skill_ids)`: registra qué habilidades co-resolvieron una consulta (máximo 6 por grupo). `select_objectives` prioriza: (0) vincular co-activaciones frecuentes (≥ 3 veces) cerrando el subgrafo con hasta 2 “puentes” de 1-salto; (1) vincular patrones ya conocidos sin colímite; (2) eliminar habilidades de nivel 0 débiles (peso máximo saliente en $(0, 0.5)$).

4.2.4 StrategicCoRegulator

(nivel 2–3, cada 100 pasos) Toma los primeros 4 “enlaces complejos” (COMPLEX) del grafo como semilla de un patrón, propone su unión $\rightarrow$ nueva capa jerárquica.

4.2.5 IntegrityCoRegulator

(transversal, cada 50 pasos) Verifica conectividad (Axioma 8.3), cobertura de pilares (Axioma 8.4), axiomas categóricos, multiplicidad (Axioma 8.2). `resolve_conflicts(propuestas)`: implementa la prioridad del Axioma 9.5. `detect_fracture`: una fractura (Def. 7.4) se detecta cuando un invariante pasa de verdadero a falso entre dos paisajes consecutivos.

4.2.6 CoRegulatorNetwork.decide(query, grafo) — el protocolo de transición global

1. `tactical.classify_query` decide RESPONSE/ASSIST (autoritativo).
2. `step(grafo)` recoge propuestas de los 4 CRs.
3. `integrity.resolve_conflicts` elige ganador — solo INTEGRITY+REPAIR_FRACTURE puede sobrescribir la clasificación táctica a REORGANIZE.
4. Confianza asignada por CR de origen: INTEGRITY 0.95, STRATEGIC 0.85, ORGANIZATIONAL 0.75, TACTICAL 0.8.
5. Guarda los resultados completos del ciclo para que `core.py` pueda leer las Options estructurales y ejecutarlas realmente (documentado como *camino muerto corregido*: antes las opciones se calculaban pero nunca se aplicaban).

4.3 memory.py — arquitectura de memoria de 4 capas (707 líneas)

- **ProceduralMemory**: “qué funcionó para este patrón de habilidades”. `add_procedure` actualiza tasa de éxito con media móvil; `get_best_procedure(pattern_id)` (argmax); `get_best_for_query(query, min_success=0.8)` (solapamiento de palabras clave $\times$ tasa de éxito $\times \log(1 + \text{invocaciones})$).
- **SemanticMemory**: conceptos-E (clases de equivalencia). `_is_e_equivalent(r1,r2)` (Def. 5.3): mismo `pattern_id`, diferencia de `success_value` ≤ 0.1 , mismo signo. `try_form_econcept`: requiere ≥ 5 registros mutuamente equivalentes (Teo. 5.4: el funtor de conceptos-E preserva colímites).
- **MESMemory**: fachada de nivel superior; `add_record` promueve automáticamente a CONSOLIDATED tras 3 consolidaciones; `save/load` implementan “Teorema 9.9: enriquecimiento monótono” — `load fusiona` en vez de sobrescribir (nunca se pierde información). **Bug histórico documentado y corregido**: antes de la corrección, los procedimientos perdían `query_text/tactic_used/lean_goal` al guardar, rompiendo silenciosamente `get_best_for_query` tras cada recarga (“0 de 1000” coincidencias antes del fix).

4.4 patterns.py — patrones y colimites (1197 líneas)

4.4.1 PatternManager

`create_pattern(component_ids, distinguished_links, graph=...)`: si se provee el grafo, construye datos completos de funtor (`index_objects`, `index_morphisms`, mapas de funtor); sin grafo produce un patrón “desnudo” (compatible hacia atrás). `are_homologous(p1,p2)` (Def. 2.5): dos patrones son homólogos si sus *campos operacionales* (morfismos salientes no-de-co-cono de sus colimites) son isomorfos. `verify_multiplicity_principle` (Axioma 8.2): se satisface syss existe un par homólogo *no* conectado por un clúster.

4.4.2 ColimitBuilder

Docstring explícito sobre la afirmación honesta: “*I es colímite de P en la subcategoría finita G_n* ”, nunca en la categoría libre infinita.

- `verify_cocone`: verifica estructuralmente que cada componente tiene morfismo de co-cono hacia el candidato, con conmutatividad si el patrón tiene datos de funtor.
- `is_join(apex, componentes, grafo)`: chequeo directo en el preorden (más rápido que el general) — explota que la categoría es delgada, así que colímite = join. Condiciones: (1) cota superior (`is_preorder_leq`); (2) minimalidad (ninguna otra cota superior es estrictamente menor). Tiene contraparte formal en `JoinColimit.lean`.
- `verify_universal_property`: verifica $\forall X \in G_n$, si X es co-cono sobre P entonces $\exists! h : \text{colímite} \rightarrow X$ — existencia y unicidad, sobre el grafo finito conocido.
- `build_colimit(pattern, grafo, verify=True)`: construye la habilidad colímite (nivel=máx(componentes)), nombre legible autogenerado tipo “A + B + C”), crea morfismos de co-cono (SPECIALIZATION), verifica co-cono y propiedad universal, **revierte** (`grafo.remove_skill`) y **lanza ValueError** si cualquiera falla.
- `complexify(grafo, patrones, verify=True)` (Teo. 2.10): vincula en lote una lista de patrones sin colímite aún.

Chapter 5

Aprendizaje por refuerzo: `nucleo/r1/`

5.1 Formulación del MDP (`mdp.py`)

- **Estado** $x = (c_L, g_{Lean}, G, h, m)$: contexto LLM (típicamente `None` en la práctica), meta Lean, `grafo.to_dict()` completo (no un vector compacto), historial (tope 10), métricas.
- **Acciones** $A = A_1 \sqcup A_2 \sqcup A_3$: `RESPONSE`, `REORGANIZE` (`add_node/add_edge/merge/split/reweight`), `ASSIST` (táctica Lean, meta).
- **Transición**: determinista dada la acción; la nueva meta Lean se toma de `action.params["new_goal"]` (provista externamente, no calculada por ningún modelo).
- **Recompensa local (distinta de `rewards.py`)**: `RESPONSE` = +1.0; `REORGANIZE` por tabla (`merge` 0.8, `split` 0.6, `add_node` 0.5, `add_edge` 0.3, `reweight` 0.1); `ASSIST` = +10 si prueba completa, +5 si se redujeron metas, +1 si táctica válida, -1 en otro caso.
- **Terminal**: solo cuando una acción `ASSIST` trae `proof_complete=True`.

`ExperienceBuffer`: buffer circular con `save/load` tolerante a archivo ausente/corrupto.

5.2 `gnn.py` — codificador GNN (`SkillGNN`)

`graph_to_pyg(grafo)`: convierte `SkillCategory` a `torch_geometric.data.Data`. Rasgos de nodo (dim 9): one-hot de pilar (4) + nivel normalizado (1) + $\log(1 + \text{in_degree})$ (1) + $\log(1 + \text{out_degree})$ (1) + participa-en-patrón (1) + está-activo (1). Rasgos de arista (dim 6): one-hot de tipo de morfismo (5, **omite IDENTITY**) + peso (1). `SkillGNN`: proyección de entrada $\rightarrow$ 3 capas `GATConv` residuales con `LayerNorm` (multi-cabeza, 4 cabezas, `edge_dim=6`) $\rightarrow$ `global_mean_pool`. `forward_nodes` devuelve embeddings por nodo (sin pooling), usados por `GNNTacticRanker` en la cascada de solucionadores Lean.

5.3 `networks.py` — `ActorCriticNetwork`

`encode_query`: bag-of-words sobre 34 palabras clave fijas (+1 bucket UNK). `encode_goal`: **pseudo-embedding determinista basado en hash** — `hash(goal)` siembra un generador, produce un vector aleatorio normalizado L2; *no* es semánticamente significativo, solo da una firma estable por cadena. Arquitectura: `gnn` (`SkillGNN`) + `query_encoder` (MLP 35 $\rightarrow$ 128 $\rightarrow$ 256) + `goal_encoder` (MLP 32 $\rightarrow$ 128 $\rightarrow$ 256) + `fusion` (concatenación simple 768 $\rightarrow$ 256, **no** atención multi-cabeza pese a lo que sugiere el diagrama de docstring) + cabezas `actor/critic`. Aproximadamente **0.55M parámetros** con `hidden_dim=256`.

5.4 agent.py — NucleoAgent y PPO (565 líneas)

5.4.1 Selección de acción: tres niveles

`select_action`: ϵ -greedy (exploración aleatoria pura con probabilidad ϵ) $\rightarrow$ si hay red neuronal, `_select_neural` $\rightarrow$ si no, `HeuristicAgent`. Dentro de `_select_neural`: **primero** consulta memoria procedimental (`get_best_for_query`); si hay coincidencia con tasa de éxito ≥ 0.8 , **omite la red neuronal por completo**. Solo si no hay memoria útil, corre el forward pass de la red; el índice de acción muestreado se mapea a [RESPONSE, REORGANIZE, ASSIST], pero los *parámetros* concretos (táctica, operación de reorganización) se delegan siempre a `HeuristicAgent` — la red solo elige el *tipo* de acción.

5.4.2 Actualización PPO con GAE

`_ppo_update`: codifica el mismo grafo actual para todo el rollout (la estructura del grafo no varía por transición, solo el texto de consulta/meta). Calcula $GAE(\lambda)$ con recursión hacia atrás ($\delta_t = r_t + \gamma V(s_{t+1})(1 - d_t) - V(s_t)$), normaliza ventajas, y ejecuta `n_epochs=4` épocas de pérdida recortada PPO (`clip_range=0.2`) + pérdida de valor (MSE) + bônus de entropía, sin mini-batching (todo el rollout en un solo batch por época).

5.4.3 Persistencia

`save(path)`: escribe `path` (JSON: config, métricas, ϵ) y, si hay red, `path+".pt"` (pesos) — de ahí `data/neural_agent.json` + `data/neural_agent.json.pt`. `load`: si `path+".pt"` existe, marca `weights_pretrained=True`, lo que permite a `CR_tac` confiar en el enrutamiento neuronal sin esperar 50 transiciones en buffer.

5.5 rewards.py — función de recompensa estructural (334 líneas)

Hallazgo relevante: este archivo *no contiene* recompensa basada en verificación Lean ni distinción aritmética/abstracta — toda su lógica es estructural/heurística sobre el grafo:

$$r = r_{task} + \lambda_1 r_{eff} + \lambda_2 r_{org} + \lambda_3 r_{emerge} + \lambda_4 r_{hier} + \lambda_5 r_{mem}$$

con $r_{eff} = -(\alpha \cdot \text{habilidades_usadas} + \beta \cdot \Delta t)$, $r_{org} = \delta \cdot \text{coherencia} + \epsilon \cdot \text{cobertura}$ (usa métricas absolutas del grafo actual, no deltas, pese a la fórmula documentada), r_{hier}/r_{mem} delegan en `grafo.get_hierarchy_score()/get_memory_score()`.

La recompensa Lean real vive en `scripts/train_gnn_ppo.py`, no en `nucleo/r1/`: la función `_lean_reward` + `_build_lean_snippet` clasifican el problema por presencia/ausencia de `\boxed{entero}` en la solución de referencia — si no hay entero encerrado, se trata como *abstracto* (ProofNet-like) y recibe 0.5 **sin invocar Lean**; si lo hay, se busca un par (a, b) con $a + b = N/a - b = N/a \times b = N$ y se genera y ejecuta un snippet `norm_num` real, con recompensa 1.0 (verificado), 0.5 (**sorry**) o 0.3 (error/timeout). Esta distinción aritmética-vs-abstracta es puramente sintáctica, no semántica.

5.6 Consumidores de `nucleo/r1/`

`core.py` (agente principal + PPO en vivo tras cada interacción + retroalimentación de usuario), `lean/solver_cascade.py` (`GNNTacticRanker`, reordena — nunca añade/quita — la cascada de tácticas por similitud coseno con embeddings de nodos-habilidad; invariante de permutación demostrado en Lean como `gnnRankTactics_perm`), `multi_agent/specialized_agent.py` (cada uno de los 14 agentes especializados envuelve su propio `NucleoAgent`). **app.py no importa `nucleo.r1` directamente** — todo pasa por `Nucleo`.

Chapter 6

Integración con Lean 4: nucleo/lean/ y MetamathProver/

6.1 client.py — invocación de Lean y LeanResult

6.1.1 Mecanismo de invocación

Pese a que el docstring del módulo menciona un “cliente bidireccional / REPL de Lake” (documentación desactualizada), la implementación real es **verificación por lotes de un solo disparo**: `subprocess.Popen(["lake", "env", "lean", archivo, "--json"])` (no un REPL persistente), elegido deliberadamente sobre `subprocess.run` para poder matar el proceso en caso de timeout. `_resolve_lake`: busca binario absoluto → PATH → `~/.elan/bin/lake[.exe]` → `/usr/local/bin/lake`.

6.1.2 Fallback HTTP

Si el binario local no está disponible, `check_code` recibe `NOT_AVAILABLE` y cae a `_verify_via_http`: `POST {code,timeout}` a la URL de la variable de entorno `LEAN_VERIFY_URL` más `/verify` — el servicio remoto `lean_api` (§6.7). Es decir: **subproceso local es la vía primaria; HTTP remoto es el respaldo** para entornos sin toolchain de Lean (p.ej. Streamlit Cloud).

6.1.3 LeanResult

`status`: `LeanResultStatus` (`SUCCESS|ERROR|TIMEOUT|INCOMPLETE|SORRY|NOT_AVAILABLE`), `goals`, `messages` (JSON crudo de Lean), `elapsed_ms`. `error_messages` lee la clave `"data"` antes que `"message"` — corrección de un bug documentado que antes cegaba todo el diagnóstico de errores.

6.1.4 Manejo de timeouts y correcciones específicas de Windows

1. `_kill_process_tree`: en Windows, usa `taskkill /F /T /PID` para matar árbol completo de procesos — un `proc.kill()` normal deja huérfano el `lean.exe` hijo.
2. Ejecuta el subproceso síncrono dentro de `loop.run_in_executor` explícitamente para evitar problemas de `ProactorEventLoop` de Windows con pipes en contextos `async`.
3. Decodificación con `errors="replace"` para sobrevivir bytes no-UTF8 de consolas Windows.

6.1.5 Normalización de código antes de cada invocación (`_normalize_code`)

Sustituye `import Mathlib` desnudo (carga ~1.58 GB de `.olean`, medido en 742s — siempre excede el timeout de 360s) por una cabecera segura estrecha (`_SAFE_HEADER`) más importaciones

temáticas activadas por palabras clave. Descarta importaciones Mathlib desconocidas (validadas contra el árbol de archivos real). `repair_imports`: ante errores de identificador/constante desconocidos, escanea el código fuente de Mathlib para encontrar el módulo declarante e inyecta el `import` faltante automáticamente (~2.2s de escaneo vs. ~15s por corrida de Lean). Reescribe una tabla de lemas obsoletos (p.ej. `Nat.factors`→`Nat.primeFactorsList`).

6.2 `solver_cascade.py` — la cascada de solucionadores

Adaptado de `lean4-skills`, inspirado en APOLLO (arXiv:2505.05758); el docstring afirma que resuelve mecánicamente 40–60% de los casos simples.

Orden	Táctica	Timeout
1	<code>rfl</code>	1 s
2	<code>simp</code>	2 s
3	<code>ring</code>	2 s
4	<code>linarith</code>	3 s
5	<code>nlinarith</code>	4 s
6	<code>omega</code>	3 s
7	<code>exact?</code>	5 s
8	<code>apply?</code>	5 s
9	<code>aesop</code>	8 s

`SKIP_ERROR_TYPES` (`unknown_ident`, `synth_implicit`, `recursion_depth`, `synth_instance`) cortocircuitan toda la cascada. `GoalAnalyzer.prioritize` reordena por: (1) táctica de dominio de `domain_default_tactic`; (2) coincidencia de patrones regex sobre el texto de la meta; (3) vecinos-táctica en el grafo de habilidades. `GNNTacticRanker` reordena además por similitud coseno con embeddings GNN (invariante de permutación demostrado en Lean).

6.3 `sorry_analyzer.py` y `sorry_filler.py`

`find_sorries_in_text`: análisis textual puro (sin invocar Lean); ignora `sorry` dentro de comentarios `--`; extrae contexto, nombre de declaración (búsqueda regex hacia atrás hasta 50 líneas), documentación cercana (`TODO|NOTE|FIXME|STRATEGY|DEPENDENCIES`).

`SorryType` con distribución empírica documentada: `MATHLIB_EXISTS` (60%), `NEEDS_TACTIC` (20%), `MISSING_STEP` (15%), `COMPLEX_STRUCTURAL` (4%), `NOVEL_RESULT` (1%). `fill_sorry_with_cascade`: primero intenta `SolverCascade.try_fill_sorry`; si falla, genera hasta 3 `ProofCandidate` (directo, basado en tácticas, automatización `simp/aesop`); si todos fallan, marca `needs_escalation=True`.

6.4 `tactics.py` / `tactics_db.py` — dos catálogos de tácticas paralelos

Redundancia documentada: existen dos bases de datos de tácticas independientes. `tactics.py::TacticMap` tiene 18 tácticas hardcodeadas con heurísticas de `matches_goal`. `tactics_db.py::TacticsDatabase` tiene 26 tácticas en 9 categorías, incluyendo automáticas específicas de dominio ausentes en la primera (`continuity`, `measurability`, `positivity`, `fun_prop`). Esta segunda es la que realmente usa `SorryFiller` vía el singleton `get_tactics_database()`.

6.5 parser.py — interpretación de salida/errores de Lean

Capa básica (**LeanParser**): parsea JSON línea-a-línea, reconoce 6 tipos de error por regex, sugiere correcciones canned. Capa extendida (**StructuredError**, inspirada en APOLLO): clasifica en 11 tipos más finos, calcula `error_hash` (SHA-256 truncado, para deduplicación en memoria MES), y define `is_cascade_compatible` — excluye exactamente los mismos tipos que `SKIP_ERROR_TYPES` de la cascada.

6.6 Formalizaciones Lean: MetamathProver/

6.6.1 Toolchain

`lean-toolchain`: `leanprover/lean4:v4.29.0-rc4` (release candidate reciente — varios comentarios en el código documentan cambios de API específicos de esta versión). `lakefile.toml`: paquete `metamath_prover`, dependencia `mathlib` sin revisión fijada (sigue el HEAD más reciente).

6.6.2 CategoryFoundations/ — fundamentos formales de la propia arquitectura del NLE

Archivo	Contenido	sorry
<code>SkillCategory.lean</code>	Grafo de habilidades como Quiver ; construcción Paths da la categoría libre.	0
<code>ColimitVerifier.lean</code>	Verificación decidible finita de colímites/joins para FinSkillGraph <code>n</code> ; alcanzabilidad acotada; corrección y completitud probadas.	0
<code>JoinColimit.lean</code>	Preórdenes son categorías delgadas; IsJoin = co-cono + propiedad universal; unicidad de joins.	0
<code>ComplexityOrder.lean</code>	Base formal de <code>graph/complexity.py</code> : equivalencia <code>join ↔ colímite</code> , Fubini para joins, orden de complejidad tipo Bellman-Ford.	2
<code>CoRegulatorNetworkFlow.lean</code>	Formaliza el sistema de prioridad de co-reguladores (Axioma 9.5), protocolo de decisión global, E-equivalencia como Setoid , corrección de la cascada de tácticas + reordenamiento GNN.	0
<code>IsColimitBridge.lean</code>	Cierra la brecha entre IsJoin propio y <code>CategoryTheory.Limits.IsColimit</code> de Mathlib.	0

Los 2 sorry de `ComplexityOrder.lean` corresponden a `cn_join_gt_component` (inducción estructural sobre el DAG de joins, argumento de terminación no trivial) y `hierarchy_well_founded` (inducción sobre la profundidad del DAG) — documentados en el propio archivo como demostrables pero pendientes.

6.6.3 Group/ y Ring/ — álgebra clásica (0 sorry confirmado en todo el árbol)

Estructura espejo Grupo/Anillo de los 4 resultados clásicos, construidos como envoltorios sobre lemas ya existentes de Mathlib (valor añadido: demostrar y ensamblar infraestructura Mathlib

con instancias concretas, no prueba desde cero):

- **Primer Teorema de Isomorfismo:** $G/\ker(f) \cong f(G)$ / $R/\ker(f) \cong f(R)$; ejemplos concretos ($\mathbb{Z}/n\mathbb{Z}$, signo de permutaciones, grupos cíclicos).
- **Segundo Teorema (Diamante):** $S/(S \cap N) \cong (S \vee N)/N$.
- **Tercer Teorema:** $(G/N)/(K/N) \cong G/K$, más Teorema de Correspondencia (4 partes).
- **Teorema del Retículo (Cuarto Teorema):** isomorfismo de orden entre subgrupos/ideales de G/N y subgrupos/ideales de G que contienen N ; preserva encuentros, uniones y normalidad.

6.7 Servicio remoto: `lean_api/main.py`

Microservicio FastAPI independiente para verificar Lean cuando no hay toolchain local (típicamente Streamlit Cloud).

- GET `/health`: sonda de vida.
- POST `/verify`: `{code, timeout}` → escribe a un archivo temporal *dentro* de `lean_project/` (para que las importaciones resuelvan contra el Mathlib empaquetado), ejecuta `lake env lean --json` vía `asyncio.create_subprocess_exec` con `wait_for(timeout)`, devuelve `stdout/stderr/returncode` crudos (mismo formato JSON-lines que el cliente local parsea).

Despliegue en Railway (`railway.json`: builder Docker, healthcheck en `/health`); `Dockerfile` propio instala `elan`+Lean 4 stable, descarga caché precompilado de Mathlib (~500 MB) vía `lake exe cache get`; imagen final ~3–4 GB. La única conexión con el cliente local es la variable de entorno `LEAN_VERIFY_URL`.

Chapter 7

Pilares fundacionales: nucleo/pillars/

7.1 base.py — la interfaz Pillar

`PillarSkill` (dataclass: `id`, `name`, `description`, `dependencies`, `metadata`) — distinto de `nucleo.types.Skill` usado por el grafo en tiempo de ejecución. `Pillar(ABC)`: métodos abstractos `get_skills()` y `validate()`; **no hay** método abstracto de “axiomas” — cada pilar concreto expresa sus axiomas como métodos ordinarios. `PillarRegistry`: registro global + tabla de traducciones entre pilares.

7.2 Los cuatro pilares concretos

- **category_theory.py** (F_{Cat}): kernel de categorías real y funcional — `Category.compose`, `identity`, `hom`; `verify_axioms()` existe pero sus chequeos de asociatividad/identidad son *stubs TODO* (siempre devuelve `True`). 8 `PillarSkill`: `cat-basics`→`functors`→`nat-trans`→`adjunctions`→`limits`, `yoneda`, `topos-basics`→`internal-logic` (conecta con F_{Log} via τ_4).
- **logic.py** (F_{Log}): `LogicSystem` (`FOL`, `SOL_STD`, `SOL_HEN`, `HOL`, `IL`); sintaxis inductiva de fórmulas/términos. `check_metalogical_properties`: tabla canónica de completitud/compacidad/Löwenheim/Skolem/semidecidibilidad por sistema. `IntuitionisticLogic.is_constructive`: heurístico de detección de patrón LEM, explícitamente marcado `TODO` como incompleto.
- **set_theory.py** (F_{Set}): `ZFCAxiom` (9 axiomas), `describe_axiom` devuelve la fórmula de primer orden literal de cada uno; `Ordinal/Cardinal` con representaciones finitas/infinitas nombradas (ω , $\aleph_0$, $\aleph_1$, $\mathfrak{c}$). Habilidad `ch` (hipótesis del continuo) marcada `independent=True`.
- **type_theory.py** (F_{Type}): `TypeSystem` sigue el Cubo de Barendregt (`STLC`...`CIC`). `CurryHoward`: traductores estáticos `lógica`↔`tipos` (`implicación`→`función`, `conjunción`→`producto`, `disyunción`→`suma`, $\forall \rightarrow \Pi$, $\exists \rightarrow \Sigma$). `translate_to_lean`: renderiza tipos como sintaxis Lean 4 literal.

Nota arquitectónica importante: estas 4 clases `Pillar` y sus listas de `PillarSkill` son una representación *paralela y en gran medida desconectada* del grafo real de habilidades. Solo se ejercitan desde `tests/` y desde `cli.py::cmd_init` (que registra las 4 en un `PillarRegistry` nuevo al hacer `nucleo init`). **core.py nunca las importa ni instancia** — en su lugar `hardcodea` 10 habilidades de nivel 0 directamente (§2.3.3).

7.3 math_domains.py — el catálogo real (162 habilidades)

Esquema `DomainSkillDef`: `id`, `name`, `description`, `pillar`, `level` (1-3), `dependencies`, `category`, `keywords` (ES+EN). Conteo verificado: **33 de nivel 1**, **34 de nivel 2**, **95 de nivel 3** (sub-ramas finas) = 162 total, en 13 categorías matemáticas + 2 categorías Lean-específicas

(`lean-tactics`: 9 habilidades `tactic-*`; `proof-strategies`: 6 habilidades `strategy-*`).

`load_math_domains(grafo)`: ordena por nivel, degradación elegante (salta habilidades cuyas dependencias aún no existen), añade morfismos `DEPENDENCY`, morfismos de traducción **inter-pilar** (`INTER_PILLAR_TRANSLATIONS`, p.ej. álgebra homológica $\rightarrow$ topología algebraica), y vincula cada habilidad de dominio a una táctica Lean por defecto según su categoría (`CATEGORY_TACTIC_SKILL`).

`_match_skills_to_query` (vive en `core.py`, no aquí): solapamiento de tokens id/nombre + coincidencia de frase completa contra `metadata["keywords"]` (peso +2, mayor que un token suelto), top 10 resultados.

Chapter 8

Integración con modelos de lenguaje: nucleo/llm/

8.1 `client.py` — cliente multi-proveedor (529 líneas)

Proveedor primario: **Anthropic Claude** (SDK oficial `anthropic`), modelo por defecto `claude-sonnet-4-6` en el código (el YAML de despliegue configura `claude-sonnet-5`). También soporta Google GenAI, Groq y DeepSeek (vía cliente compatible OpenAI). `LLMProvider`: `ANTHROPIC`, `GOOGLE`, `GROQ`, `DEEPSEEK`, `DEMO`.

Clave API: resolución config-sobre-variable-de-entorno; `_ENV_KEYS` mapea cada proveedor a su variable (`ANTHROPIC_API_KEY`, `GOOGLE_API_KEY`, `GROQ_API_KEY`, `DEEPSEEK_API_KEY`). `reconfigure()` permite intercambio en caliente de proveedor/modelo/clave, limpiando el historial conversacional si cambia el proveedor (evita mezclar formatos de mensaje incompatibles).

Streaming: el parámetro `stream` existe en la firma pero **no está implementado** — siempre se hace la llamada no-streaming, pese a que el método es `async def generate`.

Manejo de “adaptive thinking”: como el pensamiento extendido está activo por defecto en Sonnet 5/Opus 5, la respuesta puede comenzar con un bloque `ThinkingBlock` sin `.text`; el código filtra explícitamente `b.type=="text"`. También omite el parámetro `temperature` para estos modelos porque lo rechazan con HTTP 400.

Sin reintentos/backoff: cualquier excepción revierte el último mensaje de usuario añadido (evita dos mensajes consecutivos del mismo rol) y se relanza para que `process_sync()` la capture y muestre.

Modo demo: `DemoLLMClient` se activa sin clave API configurada; responde con patrones canned (saludo, “táctica” → `"simp"`, mensaje genérico de modo demo).

8.2 `prompts.py` — catálogo de plantillas (314 líneas)

10 plantillas con nombre (`PromptManager`), todas en español: `TACTIC_SUGGESTION`, `TACTIC_CHAIN`, `PROOF_EXPLANATION`, `CONCEPT_EXPLANATION`, `FORMALIZE_STATEMENT`, `FORMALIZE_PROOF_SKETCH`, `ERROR_DIAGNOSIS`, `SKILL_RECOMMENDATION`, `SKILL_PATH`, `CURRY_HOWARD_TRANSLATE`. **No hay inyección few-shot de miniF2F en este archivo** — ese mecanismo (`_build_few_shot_context`) vive directamente en `core.py`, no en `prompts.py`.

Chapter 9

Evaluación de respuestas: `nucleo/eval/`

9.1 `math_evaluator.py` (492 líneas)

Modelado explícitamente sobre la lógica de evaluación de InternLM-Math.

9.1.1 Extracción (`extract_answer`)

Orden de prioridad: (1) `\boxed{...}/\fbox{...}` con emparejamiento correcto de llaves anidadas (toma la *última* ocurrencia); (2) patrón “la respuesta es:”; (3) último literal numérico encontrado.

9.1.2 Normalización (`_normalize_string`)

Larga cadena de reglas de limpieza LaTeX: elimina `\!`, `\left/\right`, unifica `tfrac/dfrac` $\rightarrow$ `frac`, elimina unidades `\text{...}` finales, marcadores de grado, símbolos de moneda/porcentaje; corrige decimales sin cero inicial; mapea `infinity` $\rightarrow$ `\infty`; mapea `j` $\rightarrow$ `i` (convención de ingeniería para número imaginario, solo si no hay ya una `i`); repara fracciones `\frac` mal formadas y divisiones simples `a/b`.

9.1.3 Comparación (`_compare_answers`) — primera coincidencia gana

1. Coincidencia exacta de cadenas normalizadas $\rightarrow$ `"exact"`.
2. Igualdad numérica (`math.isclose`, `rel_tol=1e-4`; también prueba variantes $\times 100/\div 100$ para discrepancias porcentaje-vs-decimal) $\rightarrow$ `"numeric"`.
3. Coincidencia exacta tras eliminar corchetes/paréntesis $\rightarrow$ `"exact_cleaned"`.
4. Igualdad simbólica vía `sympy` (solo si está disponible): parsea LaTeX, simplifica la diferencia a cero, o evalúa numéricamente con tolerancia `1e-3` $\rightarrow$ `"symbolic"`.
5. En otro caso, `"none"` (incorrecto).

Expuesto en `Nucleo.evaluate_answer(prediction, reference)`, usado por `scripts/evaluate_benchmark.py` para medir precisión en MATH/GSM8K.

Chapter 10

Consultores Avanzados: nucleo/consultores

10.1 Propósito

Módulo opcional para matemáticos expertos: en vez de una respuesta conversacional, genera **N artífices verificables** en una única llamada LLM estructurada — archivo Lean autocontenido, esqueleto de prueba en lenguaje natural, script solucionador Python opcional (para subproblemas numéricos/de optimización), script “puente” que convierte `solution.json` a una instancia Lean, plan de verificación y comandos de shell exactos. Cada candidato se verifica realmente en Lean (no autorreporte del LLM), se puntúa/rankea, y la sesión completa se registra en `MESMemory`.

Reutiliza explícitamente las *mismas* instancias de `LLMClient`, `LeanClient`, `PatternManager` y `MESMemory` que el Nucleo principal — es un modo/plugin, no una pila separada.

10.2 Protocolo de marcadores (`master_prompt.py`)

Una única llamada LLM produce un bloque estructurado con marcadores `%%MARKER%%...%%END_MARKER%%`: `%%CLASIFICACION%%` (JSON tipo/subtipo/confianza) → `%%SPEC%%` (especificación formal) → `%%CANDIDATO_i%%` (para $i = 1..N$, con sub-bloques Lean/esqueleto/solver/puente/plan/-comandos) → `%%METRICAS%%` (autorreporte JSON) → `%%RESUMEN%%` → `%%AUDIT%%`. Una “**consultor**” en este vocabulario no es un agente separado que corre N veces — es un espacio de candidato dentro de una única respuesta LLM; el paralelismo real de la orquestación es solo en la verificación Lean posterior (secuencial, sin `asyncio.gather`).

10.3 `orchestrator.py` — `ConsultoresModule.process(query)`

1. Una llamada LLM → respuesta cruda con todos los marcadores.
2. Parseo de clasificación, especificación, candidatos, notas.
3. **Bucle de verificación Lean**: para cada candidato con archivo Lean no vacío, `await self._lean.check_code(...)`; recuenta `sorry_count` directamente del contenido (no confía en el autorreporte del LLM).
4. `score_and_rank` (`reranker.py`).
5. Construye `AuditTrace` con tiempos y modelo usado.
6. `_record_in_memory`: registra un `ExperienceRecord` (éxito= 1.0 si algún candidato verificó) y un procedimiento en memoria procedimental bajo `pattern_id="consultores"`.

10.4 Fórmula de puntuación (`artifacts.py::CandidateMetrics.compute`)

$$\text{total_score} = 0.25 s_{\text{syntax}} + 0.15 c_{\text{mathlib}} + 0.10 \cdot \frac{1}{\text{complejidad}} + 0.25 c_{\text{completeness}} + 0.25 s_{\text{lean}}$$

donde $s_{\text{lean}} \in \{1.0 \text{ (verificado sin sorry), } 0.5 \text{ (con sorry), } 0.0 \text{ (rechazado), } 0.25 \text{ (aún no ejecutado)}\}$
 — el resultado real de Lean domina el ranking, consistente con la filosofía “Lean es verdad”.

10.5 `solver_runner.py` — ejecución aislada

run_solver: **compila el script primero** (`compile(...,"exec")`) para capturar `SyntaxError` antes de lanzar subprocesso; ejecuta en subprocesso aislado (`subprocess.run([sys.executable,...])`, **nunca eval/exec en proceso**); lee `solution.json` generado. **run_bridge + run_full_pipeline:** encadena `solver`→puente→`lean_client.check_code` — función invocada bajo demanda por botón “Ejecutar solver” en la página de Streamlit correspondiente.

10.6 Activación

Único disparador: `pages/4_Consultores_Avanzados.py` — un *toggle* + slider de número de candidatos (1–6) en la UI, que llama `nucleo.set_consultores_mode(n_candidates)/disable_consultores_mode()`, y un formulario que dispara `nucleo._consultores.process(query)`.

Chapter 11

Sistema multi-agente: nucleo/multi-agent/

11.1 Propósito y estado real: LATENTE

Este subsistema reemplaza (opcionalmente) el agente PPO único por **14 agentes RL especializados**, uno por categoría matemática, más una jerarquía categórica adicional de “agentes colimite/pilar”. **Hallazgo crítico:** `Nucleo.set_multi_agent_orchestrator()` **no se invoca en ningún lugar de la aplicación desplegada** — ni en `app.py`, ni en ninguna página de Streamlit, ni en ningún script de producción. Es código completo, probado (vía `tests/test_domain_tactic_pipeline.py`) y funcional, pero **dormido**.

Lo único de este subsistema que SÍ está siempre activo es un par de funciones ligeras: `classify_query(texto)` (clasificador por conteo de coincidencias de palabras clave en 14 categorías, sin aprendizaje) y `domain_default_tactic(categoria)`, llamadas incondicionalmente dentro de `Nucleo._math_via_lean` (§2.3.5) para elegir una táctica inicial de la cascada.

11.2 `specialized_agent.py` — `SpecializedAgent`

14 categorías fijas (CATEGORIES). `SpecializedAgent` envuelve un `NucleoAgent` propio, con pesos por categoría en `training/agents/best/{categoria}.pt` (con fallback a los pesos base compartidos `data/neural_agent.json.pt`). `select_action`: si hay `mes_bridge` configurado, primero consulta `query_best_tactic` — si hay táctica conocida-buena, **omite la red neuronal por completo**, igual que en `NucleoAgent._select_neural`.

11.3 `orchestrator.py` — `MultiAgentOrchestrator`

`route(query)`: clasifica y obtiene/crea perezosamente el agente de esa categoría. `record_solution`: reenvía al agente y, vía `mes_bridge`, registra convergencias entre categorías.

11.4 `mes_bridge.py` — `MESBridge`, el puente hacia `nucleo/mes/`

Cada `SpecializedAgent` se describe como “un co-regulador L1 con su propia `ProceduralMemory`”. `record_success`: registra el procedimiento en la memoria de la categoría; si ≥ 2 categorías distintas resuelven *la misma* consulta (por hash normalizado) con éxito, dispara `_handle_convergence` — construye un patrón sintético y, si hay `colimit_builder/skill_graph` reales, llama `build_colimit` para crear una “habilidad emergente” de nivel L2. Cae a una `_FallbackMemory` en memoria si `nucleo.mes` no está disponible por algún motivo (degradación elegante).

11.5 pillar_agents.py y colimit_agents.py — jerarquía categórica L0→L1→L2→L3

Formalismo distinto de RL: cada PillarAgent/ColimitAgent es un *envoltorio computacional* alrededor de un **join verificado** (`is_join()`) de las habilidades de un pilar (L1, 4 agentes) o de una categoría matemática (L2, 14 agentes), más un único “orquestador” L3 que agrupa los 14 L2 (**construido con `verify=False`**, a diferencia de L1/L2). `ColimitAgent.select_tactic`: cascada de 5 niveles puramente heurística/memorística (sin red neuronal en esta capa): mejor táctica recordada → táctica mediadora cruzada → “activar co-cono” (puntuación por peso×relevancia de habilidades componentes) → “señal de pilar” (morfismos L1→L2 con palabras clave específicas por pilar) → táctica por defecto estática.

Estas dos jerarquías (**SpecializedAgent/MultiAgentOrchestrator de un lado, PillarAgent/ColimitAgent del otro**) son caminos de código independientes que nunca se conectan entre sí — solo comparten el vocabulario de `classify_query` y la tabla `CATEGORY_DEFAULT_TACTICS` (que además existe **duplicada** de forma independiente en `pillars/math_domains.py`, según un comentario explícito en ese archivo).

Bug latente documentado: `multi_agent/mes_bridge.py` contiene un `import` de `nucleo.graph.skills` (módulo **inexistente** en el árbol real de `graph/`), envuelto en un `try/except` amplio que lo silencia — el paso de “añadir habilidad emergente al grafo” es código muerto incluso si se corrigiera el `import`, porque `SkillNode` vive realmente en `graph/category.py` con una firma de constructor incompatible.

11.6 scripts/train_multiagent.py — productor de los pesos

CLI de 3 fases (rutina de **routing** supervisada; cabeza de táctica con supervisión débil por heurísticas de texto; PPO opcional con recompensa Lean real, `--with-lean`) que produce los `.pt` por categoría bajo `training/agents/`. Es el único productor de los artefactos que `SpecializedAgent` consumiría si el subsistema se activara alguna vez.

Chapter 12

Interfaz de usuario: `app.py` y `pages/`

12.1 `app.py` — estructura general (1664 líneas)

12.1.1 Arranque

Fuerza UTF-8 globalmente (necesario porque Streamlit Cloud corre en Linux con locale ASCII); reconfigura todos los `StreamHandler` de logging a UTF-8; silencia librerías ruidosas a `WARNING` mientras deja `httpx` en `INFO` (única señal de que el LLM se llamó de verdad). `_explica_error_proveedor()`: traduce errores crudos de API (saldo agotado, clave inválida, 429, 529 sobrecargado) a mensajes en español.

12.1.2 Instanciación del Nucleo (punto clave)

```
@st.cache_resource(show_spinner="Iniciando Nucleo Logico Evolutivo...")
def _init_nucleo():
    cfg = NucleoConfig.from_yaml("nucleo_config.yaml") si existe
    n = Nucleo(cfg)
    # corre n.initialize() en hilo daemon con loop propio
    # (evita chocar con el loop de asyncio de Streamlit)
    t.join(timeout=20)
    return n, ""
```

`@st.cache_resource` garantiza **una sola instancia de Nucleo por proceso**, compartida por todas las páginas (que la reobtienen buscando `_get_nucleo` en `sys.modules`). `_arranca_warmup_lean()`: precalienta Mathlib **al abrir la app** (no al primer mensaje), en hilo daemon, con centinela `.lean_warm`.

12.1.3 Flujo de chat: entrada del usuario → Nucleo → render

```
prompt = st.chat_input(...)
nucleo = _get_nucleo()
if consulta_matematica y warmup en curso:
    hilo_warm.join(timeout=1140)
```

```
nucleo.reconfigure_llm(proveedor, modelo, api_key, max_tokens)

nucleo.sync_conversation(historial)

nr = nucleo.process_sync(prompt)          # LLAMADA PRINCIPAL

vd = nucleo.get_viz_data(prompt)          # datos para Visualizaciones

# render de nr.content, confianza, metadata

# botones +1.0 / -0.5 -> nucleo.apply_feedback()
```

12.1.4 Métodos de Nucleo usados por la UI (superficie pública real)

Nucleo(config), initialize(), _is_mathematical(), reconfigure_llm(), sync_conversation(), process_sync(), get_viz_data(), apply_feedback(), _lean (acceso directo, página Consultores), consultores_active/set_consultores_mode()/disable_consultores_mode()/_consultores.process_stats.

12.1.5 Navegación multipágina

st.navigation registra page_home (chat, por defecto) + 4 páginas: 1_Visualizaciones.py, 2_Verificador.py, 3_Instalar_Lean.py, 4_Consultores_Avanzados.py.

12.2 Páginas

- **1_Visualizaciones.py** (1957 líneas): grafo categórico (estático de respaldo con 76 habilidades codificadas a mano, o en vivo desde `nucleo.get_viz_data()`), proyección t-SNE/PCA de embeddings, diagrama de arquitectura, panel de complejización MES (patrón→colímite→grafo complejizado).
- **2_Verificador.py** (614 líneas): carga `.txt/.tex/.pdf` con una demostración o conjetura; 4 modos (verificar/evaluar/solo-formalizar/detectar-errores); reutiliza la instancia cacheada de Nucleo.
- **3_Instalar_Lean.py** (419 líneas): guía de instalación de Lean 4 por SO, sin dependencia de Nucleo; botón de comprobación real (`lean --version`, `lake --version`, prueba `norm_num`).
- **4_Consultores_Avanzados.py** (569 líneas): ver Capítulo 10.

Chapter 13

Infraestructura y despliegue

13.1 Dockerfile

Imagen autocontenida (`python:3.11-slim` + `elan/Lean 4 stable` + Mathlib precompilado vía `lake exe cache get` + Streamlit), capas ordenadas para maximizar caché de Docker (cabecera de dependencias Lean copiada antes que el código de la app). `HEALTHCHECK` sobre `/_stcore/health`, puerto 8501.

13.2 deploy/

`README_DEPLOY.md` compara plataformas: Vultr Guadalajara (~\$24/mes, recomendado para LATAM), DigitalOcean São Paulo, Hetzner, HuggingFace Spaces (sin Lean), RunPod (GPU); marca explícitamente **Streamlit Cloud como no viable** (1 GB RAM insuficiente). `setup_vps.sh`: instalación Ubuntu completa con servicio `systemd` + reverse-proxy Nginx.

13.3 Arranque local en Windows (autoarranque)

Cadena: acceso directo → `Metamatematico.vbs` (lanza sin consola visible) → `Metamatematico.ps1` → `env_local.ps1` (redirige todos los cachés/temporales a disco E: por espacio en C:) → Streamlit en `localhost:8501`. Variante `launch_local.ps1` para el Programador de Tareas, mata cualquier instancia previa y relanza sin abrir navegador.

13.4 nucleo_config.yaml

Configuración YAML de producción que sobrescribe los valores por defecto de `config.py`: `lean.timeout_ms=360000`, `llm.model="claude-sonnet-5"`, `graph.gnn_hidden_dim=256`.

Chapter 14

Scripts de entrenamiento y pipeline de datos

14.1 Pipeline de datos

`prepare_training_data.py` → `balance_datasets.py` → `split_by_category.py`: cadena de scripts que descargan/clasifican/balancean problemas de MATH, GSM8K, NuminaMath, ProofNet y ~14 datasets adicionales por categoría matemática (con extractores dedicados por dataset y manejo explícito de datasets ausentes vía [SKIP]), produciendo splits 80/10/10 en `training/by_category/`.

`seed_from_datasets.py`: conecta datasets descargados a la memoria de producción — convierte 316 teoremas de ProofNet en `ExperienceRecords` (éxito=1.0), parsea 157 teoremas de miniF2F como banco few-shot categorizado (`data/lean_examples.json`, el mismo que usa `_build_few_shot_context` en `core.py`).

14.2 Entrenamiento

`train_gnn_ppo.py`: Fase 1 preentrenamiento supervisado (enrutamiento hacia ASSIST); Fase 2 opcional PPO con recompensa Lean real (`_lean_reward`, §5). Guarda en `data/neural_agent.json.pt` (consumido en producción por `NucleoAgent.load`) y checkpoints en `E:/checkpointsmetamatematico/`.

`train_multiagent.py`: entrena los 14 agentes especializados (3 fases, §11) — productor de pesos para un subsistema actualmente inerte en la app desplegada.

14.3 Evaluación y herramientas

`evaluate_benchmark.py`: evalúa el NLE completo (no solo el LLM) contra MATH/GSM8K, registrando tanto accuracy como señales internas independientes del LLM (cobertura del grafo, fuente de decisión CR, verificación Lean, aciertos de memoria MES). `genera_mapa_sistema.py` / `generate_hierarchy_pdf.py` / `generate_pdf.py`: generan PDFs de documentación honesta del propio sistema (con `fpdf/matplotlib`), incluyendo explícitamente el estado ACTIVO/PARCIAL/INERTE por componente. `local_agent.py`: agente que el usuario ejecuta en su propia máquina para ampliar la capacidad de verificación Lean de una instancia web remota (modo `--server` con polling HTTP, infraestructura preparada pero endpoints servidor aún no implementados).

Chapter 15

Flujo end-to-end de una consulta matemática

A modo de síntesis, se traza una consulta típica (“demuestra que $\sqrt{2}$ es irracional”) a través de todo el sistema:

1. **UI** (`app.py`): el usuario escribe en el chat; se llama `nucleo.process_sync(prompt)`.
2. **Orquestador** (`core.py::process`): actualiza estado, invoca `CoRegulatorNetwork.decide()`.
3. **MES** (`co_regulators.py`): `TacticalCoRegulator.classify_query` detecta palabras clave matemáticas $\rightarrow$ `RESPONSE` (que luego se enruta internamente a Lean por ser matemático) o `ASSIST` directo; `IntegrityCoRegulator.resolve_conflicts` confirma la decisión.
4. **Multi-agente (ligero)** (`specialized_agent.py`): `classify_query` $\rightarrow$ “number-theory”; `domain_default_tactic` $\rightarrow$ p.ej. `norm_num`.
5. **LLM** (`llm/client.py`): formaliza el enunciado en Lean 4, anclado a referencias Mathlib verificadas, con contexto few-shot de miniF2F.
6. **Lean** (`lean/client.py`): `check_code()` ejecuta `lake env lean --json` (o cae a `lean_api` remoto); si hay `sorry`, `SolverCascade/SorryFiller` intentan cerrarlo con la cascada de 9 tácticas, reordenada por `GoalAnalyzer/GNNTacticRanker`.
7. **LLM** (de nuevo): traduce el Lean verificado a explicación en lenguaje natural, fiel al código real.
8. **MES**: se registra un `ExperienceRecord`, se intenta formar un E-concepto, se aprende un procedimiento si éxito > 0.3 .
9. **RL** (`rl/agent.py`): se construye una `Transition` y se aplica una actualización PPO en vivo (aprendizaje continuo, opcionalmente autoguardado).
10. **Evolución** (`graph/evolution.py`): si algún CR propuso una unión/reorganización estructural, se aplica *después* de resolver la consulta actual.
11. **UI**: se renderiza la respuesta con insignia de verificación, y se guardan datos de visualización (`get_viz_data`) para la página de grafo/embeddings.

Chapter 16

Estado real del sistema: honestidad y limitaciones conocidas

Esta sección reúne, de forma centralizada, todas las brechas entre la aspiración documentada en docstrings/papers y el comportamiento verificado del código, tal como quedó expuesto en cada capítulo:

- **Colapso del agente neuronal** (`rl/`, `mes/co_regulators.py`): los pesos entrenados de `NucleoAgent` colapsaron a devolver siempre la acción `ASSIST`, independientemente de la entrada. `TacticalCoRegulator` detecta esto en tiempo de ejecución (sondeo con 4 consultas heterogéneas) y cae automáticamente a la heurística de palabras clave — el sistema es robusto a este fallo, pero en la práctica el enrutamiento neuronal no gobierna las decisiones.
- **Multi-agente completamente dormido**: `Nucleo.set_multi_agent_orchestrator()` y `MultiAgentOrchestrator(...)` nunca se invocan fuera de sus propios módulos/tests. Solo las funciones ligeras `classify_query/domain_default_tactic` están siempre activas.
- **Pilares formales desconectados**: las 4 clases `Pillar` (`ZFC/Cat/Lógica/Tipos`) con sus `PillarSkill` son ejercitadas solo por `cli.py init` y tests; el grafo real en producción usa `math_domains.py` + 10 habilidades hardcodeadas en `core.py`.
- **Dos catálogos de tácticas Lean paralelos** (`tactics.py` de 18 tácticas, `tactics_db.py` de 26) — solo el segundo alimenta `SorryFiller` en la práctica.
- **Tabla `CATEGORY_DEFAULT_TACTICS` duplicada** de forma independiente en `multi_agent/colimit_agents.py` y `pillars/math_domains.py` (documentado como tal en un comentario del segundo).
- **Import roto en `multi_agent/mes_bridge.py`**: referencia a `nucleo.graph.skills` (módulo inexistente), silenciada por un `try/except` amplio; el paso de “añadir habilidad emergente al grafo” es código muerto.
- **2 sorry pendientes en `ComplexityOrder.lean`** (bien-fundación/terminación del orden de complejidad) — el único archivo `.lean` no completamente probado en todo el árbol `MetamathProver/`.
- **`verify_axioms()` de `category_theory.py`** (pilar formal, no el grafo real) tiene sus chequeos de asociatividad/identidad como stubs `TODO` que siempre devuelven `True`.
- **Streaming LLM no implementado**: el parámetro `stream` existe en la firma de `generate()` pero nunca se usa.
- **`encode_goal` no es semánticamente significativo**: es un pseudo-embedding determinista basado en `hash()`, no aprendido ni relacionado con similitud semántica real.

- **Asimetría de ida-y-vuelta en `NucleoConfig.to_dict()`:** omite `fracture_check_interval`, `max_repair_attempts` y `live_learning_autosave`.
- **Fusion por concatenación, no atención multi-cabeza:** el diagrama de arquitectura documentado en `rl/__init__.py` sugiere “Multi-Head Attention” pero `ActorCriticNetwork.fusion` es una simple concatenación + MLP.
- **Streamlit Cloud explícitamente no viable** para despliegue (1 GB RAM insuficiente para Lean+Mathlib) según la propia documentación de despliegue; el camino remoto de verificación (`lean_api` en Railway) existe precisamente para cubrir ese caso.

Estas limitaciones no son errores ocultos: la mayoría están documentadas explícitamente en comentarios o docstrings del propio código (“TODO”, notas de diseño, mensajes de log de advertencia), lo que refleja una cultura de desarrollo que prioriza honestidad sobre la aspiración del paper — consistente con la nota de memoria del proyecto: “*Claim honesto: soy colimite de P en la subcategoría delgada finita G_n* ”, nunca una afirmación más fuerte de la que el código pueda respaldar.

Chapter 17

Conclusión

NLE v7.0 es un sistema de mayor alcance que lo que sugiere su interfaz de chat: bajo una única clase orquestadora (**Nucleo**) coexisten un grafo categórico con evolución temporal formalizada, una red de cuatro co-reguladores que sustituye a un agente de RL monolítico, un agente GNN+PPO que sigue aprendiendo en vivo pese a haber colapsado en producción, una integración con Lean 4/Mathlib tratada como única fuente de verdad matemática (con fallback local/remoto y una cascada de nueve tácticas automatizadas), cuatro pilares fundacionales formales, un catálogo de 162 habilidades de dominio, un módulo de generación de múltiples candidatos verificables para uso experto, y una jerarquía multi-agente de 14 especialistas que existe, funciona y está probada, pero permanece sin activar en el camino de producción. El propio núcleo formal en Lean (**MetamathProver**/) certifica, con solo dos **sorry** pendientes en todo el árbol, las propiedades categóricas que el código Python asume en tiempo de ejecución — un raro caso en que la implementación y su especificación formal conviven en el mismo repositorio y se verifican mutuamente.